Python For Loops

for Loopچیست؟

از حلقه forبرای پیمایش (**Iterate**) روی اعضای یک مجموعه یا یک شیء قابل پیمایش (**Iterable**) استفاده می‌کنیم

می‌توانیم روی موارد زیر forاجرا کنیم:

List، Tuple ، Set ، Dictionary ،String، Range

ساختار کلی:

```
for variable in iterable:
    # code
```

یعنی برای هر عضو موجود در iterable، کد داخل حلقه را اجرا کن

مثال باList

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

خروجی:

```
apple
banana
cherry
```

در هر بار اجرای حلقه، متغیر fruitیک مقدار جدید دریافت می‌کند:

دور اول → fruit = "apple"

دور دوم → fruit = "banana"

دور سوم → fruit = "cherry"

نکته مهم

در forلازم نیست قبل از حلقه متغیر شمارنده تعریف کنیم در whileمعمولاً می‌نویسیم:

```
i = 1
```

```
while i <= 5:
    print(i)
    i += 1
```

اما در for:

```
for i in range(1, 6):
    print(i)
```

خود forمقدار iرا در هر iteration تغییر می‌دهد

Loopروی String

Stringنیز قابل پیمایش است

```
word = "Python"
for letter in word:
    print(letter)
```

خروجی:

```
P
y
t
h
o
n
```

یعنی هر بار یک **character** در متغیر letterقرار می‌گیرد

مثال کاربردی String

تعداد حروف یک کلمه را با forبررسی کنیم:

```
word = "banana"
```

```
for letter in word:
    print("Letter:", letter)
```

خروجی:

```
Letter: b
Letter: a
Letter: n
Letter: a
Letter: n
Letter: a
```

Loopروی Set

از آنجا که setنیز iterable است، می‌توانیم روی آن forاجرا کنیم:

```
fruits = {"apple", "banana", "cherry"}
for fruit in fruits:
    print(fruit)
```

ترتیب خروجی Set تضمین‌شده نیست

Loopروی Tuple

```
colors = ("red", "green", "blue")
for color in colors:
    print(color)
```

خروجی:

```
red
green
blue
```

Loopروی Dictionary

در Dictionary اگر مستقیماً forاستفاده کنیم، روی کلیدها پیمایش می‌کنیم:

```
student = {
   "name": "Ali",
   "age": 20,
   "grade": 18
}
for x in student:
   print(x)
```

خروجی:

```
name
age
grade
```

دریافتValue های Dictionary

برای دریافتValue ها از ()valuesاستفاده می‌کنیم:

```
student = {
   "name": "Ali",
   "age": 20,
   "grade": 18
}
for value in student.values():
   print(value)
```

خروجی:

```
Ali
20
```

18

دریافت Key و Value

با items()می‌توانیم Key و Value را همزمان دریافت کنیم:

```
student = {
   "name": "Ali",
   "age": 20,
   "grade": 18
}
for key, value in student.items():
   print(key, ":", value)
```

خروجی:

```
name : Ali
age : 20
grade : 18
```

این روش برای کار با Dictionary بسیار مهم است

range()

یکی از مهم‌ترین ابزارها در کنار for، تابع range()است

مثلاً:

```
for i in range(5):
   print(i)
```

خروجی:

```
0
1
2
```

3

4

نکته مهم

range(5)شامل 5نیست یعنی:

$0 \le i < 5$

range(start, stop)

می‌توانیم نقطه شروع را مشخص کنیم:

```
for i in range(2, 6):
   print(i)
```

خروجی:

2

3

4

5

عدد 6شامل نمی‌شود

range(start, stop, step)

پارامتر سوم مقدار حرکت را مشخص می‌کند:

```
for i in range(2, 11, 2):
   print(i)
```

خروجی:

2

4

6

8

10

یعنی:

start = 2

stop = 11

step = 2

شمارش معکوس

می‌توانیم از stepمنفی استفاده کنیم:

```
for i in range(10, 0, -1):
   print(i)
```

خروجی:

10

9

8

7

6

5

4

3

2

1

این روش برای Countdown بسیار کاربردی است

break

breakباعث می‌شود کل حلقه متوقف شود

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
  print(fruit)
  if fruit == "banana":
    break
```

خروجی:

```
apple
banana
```

وقتی به bananaرسیدیم:

```
break
```

اجرا می‌شود و حلقه تمام می‌شود

جای breakمهم است

این دو برنامه با هم تفاوت دارند

حالت اول

```
for fruit in fruits:
  print(fruit)
  if fruit == "banana":
    break
```

خروجی:

```
apple
banana
```

حالت دوم

```
for fruit in fruits:
  if fruit == "banana":
    break
  print(fruit)
```

خروجی:

```
apple
```

چون قبل از print، break اجرا شده است

continue

continue فقط iteration فعلی را رد می‌کند

مثلاً نمی‌خواهیم banana چاپ شود:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
  if fruit == "banana":
    continue
  print(fruit)
```

خروجی:

```
apple
cherry
```

مثال for + if

تمام اعداد زوج بین ۱ تا ۱۰:

```
for i in range(1, 11):
  if i % 2 == 0:
    print(i)
```

خروجی:

```
2
```

4

6

8

10

elseدر For Loop

Pythonاجازه می‌دهد بعد از forاز elseاستفاده کنیم:

```
for i in range(5):
    print(i)
else:
    print("Loop finished")
```

خروجی:

```
0
1
2
3
4
Loop finished
```

یعنی وقتی Loop به صورت طبیعی تمام شود، elseاجرا می‌شود

breakو else

اگر Loop با breakمتوقف شود، elseاجرا نمی‌شود:

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

```
else:
    print("Loop finished")
```

خروجی:

```
0
1
2
```

Loop finishedچاپ نمی‌شود

Nested For Loops

یعنی یک forداخل forدیگر

مثلاً:

```
colors = ["red", "blue"]
fruits = ["apple", "banana"]
for color in colors:
    for fruit in fruits:
        print(color, fruit)
```

خروجی:

```
red apple
red banana
blue apple
blue banana
```

مثال کاربردیNested Loop

چاپ جدول ضرب:

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j)
```

البته برای نمایش زیباتر جدول:

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()
```

pass

اگر به هر دلیلی بخواهیم یک forخالی داشته باشیم، Pythonخطا می‌دهد

مثلاً:

```
for i in range(5):
```

خطای Syntax/Indentation خواهد داشت

می‌توانیم از passاستفاده کنیم:

```
for i in range(5):
    pass
```

passیعنی فعلاً هیچ کاری انجام نده

مقایسه forو while

while	for
مناسب اجرای حلقه تا برقرار بودن شرط	مناسب پیمایش مجموعه‌ها
تعداد iteration ممکن است نامشخص باشد	معمولاً تعداد iteration مشخص است
بیشتر بر اساس یک شرط کار می‌کند	با List, Tuple, Set, String و... بسیار مناسب
معمولاً به متغیر کنترل‌کننده نیاز دارد	range()بسیار پرکاربرد است
خطر Infinite Loop بیشتر	خطر Infinite Loop کمتر

مثلاً وقتی یک List داریم:

```
students = ["Ali", "Sara", "Reza"]

for student in students:
    print(student)
```

forانتخاب طبیعی‌تری است

اما:

```
password = ""

while password != "1234":
    password = input("Password: ")
```

اینجا whileمناسب‌تر است، چون نمی‌دانیم کاربر چند بار تلاش خواهد کرد

تمرین ۱

با forاعداد 1تا 10را چاپ کنید

تمرین ۲

با استفاده از ()rangeاعداد زیر را چاپ کنید:

2

4

6

8

10

12

14

16

18

20

تمرین ۳

با forروی List زیر پیمایش کنید:

names = ["Ali", "Sara", "Reza", "Maryam"]

و نام هر دانش‌آموز را چاپ کنید

تمرین ۴

رشته زیر را پیمایش کنید:

word = "Python"

و هر حرف را در یک خط چاپ کنید

تمرین ۵

با استفاده از forو ifتمام اعداد فرد بین 1و 20را چاپ کنید

تمرین ۶

برنامه‌ای بنویسید که اعداد 1تا 10را چاپ کند، اما عدد 5را چاپ نکند

حتماً از continue استفاده کنید

تمرین ۷

برنامه‌ای بنویسید که اعداد 1تا 10را چاپ کند، اما وقتی به 7رسید، حلقه را متوقف کند

از break استفاده کنید

تمرین ۸

برنامه‌ای بنویسید که مجموع اعداد 1تا 100را با forمحاسبه کند

راهنمایی:

```
total = 0
for i in range(1, 101):
    total += i
```

```
print(total)
```

تمرین ۹ – جدول ضرب

یک عدد از کاربر دریافت کنید و جدول ضرب آن را از 1تا 10چاپ کنید

مثلاً:

Enter number: 7

7 x 1 = 7

7 x 2 = 14

7 x 3 = 21

...

7 x 10 = 70

تمرین ۱۰ – پیدا کردن عدد

Listزیر را داریم:

```
numbers = [10, 25, 7, 40, 15, 8]
```

برنامه‌ای بنویسید که بررسی کند آیا 7در List وجود دارد یا خیر

تمرین ۱۱ – بزرگ‌ترین عدد

بدون استفاده از ()maxبرنامه‌ای بنویسید که بزرگ‌ترین عدد List زیر را پیدا کند:

```
numbers = [12, 45, 7, 89, 23, 56]
```

خروجی:

89

تمرین ۱۲ – شمارش حروف

برنامه‌ای بنویسید که تعداد حروف aرا در کلمه زیر پیدا کند:

```
word = "banana"
```

خروجی:

3

راهنمایی:

```
count = 0
for letter in word:
   if letter == "a":
      count += 1
print(count)
```

تمرین چالشی Student Management –

Listزیر را داریم:

```
students = ["Ali", "Sara", "Reza", "Maryam", "Hassan"]
```

برنامه‌ای بنویسید که:

۱. تمام دانش‌آموزان را چاپ کند
۲. شماره هر دانش‌آموز را نیز نمایش دهد
۳. بررسی کند آیا "Reza"در List وجود دارد
۴. تعداد دانش‌آموزان را محاسبه کند
۵. نام‌هایی که با حرف "A"شروع می‌شوند را نمایش دهد

پروژه کوچک Number Analyzer –

یک List از اعداد داشته باشید:

```
numbers = [12, 7, 20, 15, 8, 31, 40, 5]
```

با استفاده از forبرنامه‌ای بنویسید که:

- تعداد اعداد را مشخص کند
- مجموع اعداد را محاسبه کند
- اعداد زوج را نمایش دهد

- اعداد فرد را نمایش دهد
- بزرگ‌ترین عدد را پیدا کند
- کوچک‌ترین عدد را پیدا کند

محدودیت :برای بخش پیدا کردن بزرگ‌ترین و کوچک‌ترین عدد، فعلاً از ()maxو ()minاستفاده نکنید